01 / MODEL AND SCOPE

Comparing anime and movies locally

Goal. Represent each of 1,000 anime and 1,000 movies as a dense text vector, then score every anime–movie pair. This gives $1{,}000 \times 1{,}000 = 1{,}000{,}000$ comparisons. The workflow is independent of the project's Flask/D3 dashboard and reads the existing collector CSVs.

Selected model: `Qwen/Qwen3-Embedding-0.6B`. This is a free, downloadable text embedding model under Apache 2.0. It supports more than 100 languages, a 32K-token context, and up to 1,024 embedding dimensions [1,2]. The script uses the full 1,024 dimensions. Its size makes it a practical candidate for local desktop inference. No hosted embedding service or API payment is required.

Selection rationale. General embedding benchmarks help identify strong candidates, but do not establish which model works best on these specific plots and metadata. This is a practical baseline, not a claim of a universal or current leaderboard winner. Validate the rankings on human-reviewed anime–movie pairs before choosing thresholds.

Candidate	Practical tradeoff
Qwen3-Embedding-0.6B	Selected: multilingual, Apache 2.0, moderate model size.
BAAI/BGE-M3	Free MIT alternative; 1,024 dimensions and 8,192-token context [3].
Google EmbeddingGemma	Smaller 300M alternative; 768 dimensions, 2,048-token context; requires Gemma license acceptance for model access [4].
Qwen3-Embedding-8B	Larger family option with greater local memory and computation requirements [1].

Reproducible model choice. The default is pinned to this repository commit, checked on 11 October 2026:

```
97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3
```

The run manifest records the requested and resolved revisions, actual package versions, input hashes, device, and token budget. Changing any of these can change scores; hardware differences can cause small numerical differences.

Input files. The defaults are `output/anime_top_1000.csv` and `output/movies_top_1000.csv`, relative to `data/semantic_similarity/`. Both inspected files contain 1,000 rows. The script requires that count by default and validates unique (source, source_id) keys. It retains input row order.

02 / TEXT REPRESENTATION

What becomes a dense vector

Each row produces one labeled document. Only the following fields enter the embedding model; provenance stays in separate record maps.

CSV field	Preparation and purpose
`title`	Preferred display title; contributes names and contextual clues.
`genres`	Parse the JSON array and join cleaned genre names with commas.
`tags`	Parse the JSON array and join AniList tags or TMDB keywords.
`synopsis`	Clean plot description; contributes narrative, setting, and themes.

Illustrative document (not an actual measured match):

```
Title: Example Story
Genres: Action, Science Fiction
Tags: Survival, Space Travel, Found Family
Synopsis: A stranded crew forms unlikely alliances
while searching for a way home.
```

Title and encoding policy. Read CSV bytes strictly as UTF-8, allowing a UTF-8 byte-order mark. Prefer `title`; never embed `title_original` or `title_native`. If the preferred title is empty or contains an obvious replacement character or a non-whitespace control character, try `title_english`, then `title_romaji`. If none is usable, omit the title field and record that decision. Valid Japanese or other non-Latin text is preserved. Valid UTF-8 that already contains mojibake cannot always be detected; the script does not guess a repair.

Cleaning. Remove HTML markup, decode HTML entities, normalize Unicode to NFC, collapse whitespace, and deduplicate genres/tags within each field using case-insensitive keys while preserving source order. Malformed JSON arrays fail validation. Missing fields contribute no label or invented text. A row with no usable embedding content fails validation.

The inspected anime file has no missing embedding fields. In the movie file, four rows lack genres, 26 lack tags, and three lack synopsis. These rows remain in the comparison; their available fields determine their vectors. The manifest and record maps make those gaps visible.

Excluded information. Popularity, ratings, release dates, source labels, IDs, and alternative titles do not enter the document. The script rebuilds the document from the four requested fields instead of trusting the existing `semantic_text` column. Spoiler-tag filtering already applied by the collectors is retained.

Length policy. Count tokens before embedding. The default budget is 2,048 tokens, including special tokens. Overlong inputs stop the run unless `-allow-truncation` is explicitly supplied. Alternatively increase `-max-tokens`, up to 32,768. Permitted truncation drops tokens from the right and marks affected rows. The record map saves full pre-truncation text.

03 / EMBEDDINGS AND SCORING

Cosine similarity for every pair

Let x_i be the cleaned document for anime i and y_j the document for movie j. The same model f_θ encodes both:

$$a_i = f_\theta(x_i) \in \mathbb{R}^{1024}, \qquad b_j = f_\theta(y_j) \in \mathbb{R}^{1024}.$$

The Sentence Transformers model configuration pools the final non-padding token representation into a single vector. Neither corpus is assigned a special retrieval-query role. Both use `model.encode(..., prompt="")` with the same preprocessing. This explicitly symmetric document encoding is a design choice for this baseline; retrieval instructions could be evaluated separately.

Normalize each vector. Divide by its Euclidean length:

$$\widehat{a}_i = \frac{a_i}{\|a_i\|_2}, \qquad \widehat{b}_j = \frac{b_j}{\|b_j\|_2}, \qquad \|a_i\|_2 = \sqrt{\sum_{k=1}^{1024} a_{ik}^2}.$$

The script requests normalized embeddings, casts to float32, checks dimensions and finite values, rejects zero vectors, and normalizes again after conversion. Saved vectors have unit length to floating-point precision.

Calculate raw cosine similarity.

$$s_{ij} = \cos(a_i, b_j) = \frac{a_i^\top b_j}{\|a_i\|_2 \|b_j\|_2} = \widehat{a}_i^\top \widehat{b}_j.$$

Stack unit vectors into $A \in \mathbb{R}^{1000\times1024}$ and $B \in \mathbb{R}^{1000\times1024}$. Then calculate all pairs together:

$$\boxed{S = AB^\top \in \mathbb{R}^{1000\times1000}}.$$

Row i follows the anime CSV; column j follows the movie CSV. Thus S_{ij} refers to the corresponding rows in the two saved record maps. Reverse-direction rankings use $S^\top$, not a second model pass.

Interpretation. Cosine lies in $[-1, 1]$. A larger score indicates closer vector directions. A score of 1 means identical directions; 0 means orthogonal directions; a negative score means opposing directions in this model space. These mathematical descriptions do not establish human notions of unrelatedness or narrative opposition. The implementation clips only numerical excursions beyond the cosine range and retains negative scores.

Why cosine? Normalization removes vector magnitude from the comparison and makes dense-embedding ranking efficient [5]. It is a standard baseline for semantic similarity. Crucially, $s = 0.8$ is not an 80% probability, rating, or fraction of shared plot. Scores are not calibrated across different models.

Field influence. One combined document gives one combined vector. There are no explicit title/genre/tag/synopsis weights. Length, vocabulary, and model attention affect influence. A weighted per-field method would be a separate experiment that requires its own evaluation.

04 / LIBRARIES AND EXECUTION

A fully local Python workflow

Library	Role
Sentence Transformers	Load the trained embedding pipeline; batch encoding and normalization.
Transformers	Qwen architecture, tokenizer, and pretrained model configuration.
PyTorch	Local model inference on CPU, NVIDIA CUDA, or Apple MPS.
NumPy	Float32 vector arrays, matrix multiplication, sorting, and `.npy` storage.
Hugging Face Hub	Initial free model download and local caching; no hosted inference.
Python standard library	CSV/JSON/gzip, Unicode and HTML cleanup, hashes, CLI, and metadata.

Dependencies. A new `requirements-embeddings.txt` keeps the analysis separate from the existing backend requirements. It specifies bounded compatible versions; the run manifest records exact installed versions. Python 3.10 or newer is required. No OpenAI library or paid API credential is used. An internet connection is needed for package installation and the initial model download; later runs can load cached weights with `-offline`.

Commands for the user, from the repository root:

```
python3 -m venv /tmp/anime-similarity-venv
source /tmp/anime-similarity-venv/bin/activate
python -m pip install -r \
  data/semantic_similarity/requirements-embeddings.txt
python data/semantic_similarity/compute_similarity.py
```

The script chooses CUDA if available, then Apple MPS, then CPU. It uses float32 inference, left padding, eager attention, evaluation mode, and batches of eight by default. If memory is insufficient, reduce `-batch-size`; use `-device cpu` if the selected accelerator is unsupported. There is no FlashAttention requirement.

Resource scale. The downloaded BF16 weights are roughly 1.2 GB; float32 weights require roughly 2.4 GB before activations and runtime overhead. Allow additional RAM and disk space. The two saved 1,024-dimensional vector arrays need approximately $2 \times 1000 \times 1024 \times 4 = 8.192$ MB. The score matrix needs $1000 \times 1000 \times 4 = 4$ MB. CSV and JSONL add further space. These are decimal MB estimates, not measured peak memory or timing.

Safe outputs. Input files are read-only. An existing destination directory is refused; select a new `-output-dir` for each run. Output files are created exclusively. `run_metadata.json` is written last with `status=complete`; its absence indicates an incomplete bundle. Keep any partial outputs for inspection and retry in a new directory.

Authoring status. The similarity script was prepared for inspection and has not been executed. Embeddings and scores will be produced when the user runs it. This PDF was compiled separately from the LaTeX source.

05 / OUTPUTS AND VALIDITY

Saved results and how to review them

Default output directory: `data/semantic_similarity/output/similarity/`.

File	Contents
`anime_embeddings.npy`	Anime unit vectors, float32.
`movie_embeddings.npy`	Movie unit vectors, float32.
`cosine_similarity.npy`	Full anime-row/movie-column matrix.
`anime_records.jsonl`, `movie_records.jsonl`	Row maps, IDs, titles, cleaned fields, exact text, token counts, quality flags.
`all_pairs.csv.gz`	All one million scores with source IDs and row indices.
`top_movies_per_anime.csv`	Best 20 movies for each anime.
`top_anime_per_movie.csv`	Best 20 anime for each movie.
`run_metadata.json`	Revisions, hashes, versions, settings, score statistics, completion status.

Array indices start at zero; ranks start at one. Top lists sort by descending cosine and resolve exact ties in candidate CSV order. `-top-k` changes only those lists. The complete matrix and complete compressed pair table are always retained. CSV scores use nine significant digits to preserve float32 values when read back.

Review plan. After running, inspect representative top matches, random pairs, and low-score pairs, including records with missing fields. Assign human similarity labels using a consistent definition such as shared themes, setting, and narrative conflict. Use these labels to compare rankings and set any threshold on a held-out subset. Do not select a threshold solely because its numerical value looks intuitive.

Limits. Genres and tag vocabularies differ across AniList and TMDB. Shared metadata, generic plot wording, or recognizable names may inflate scores. Missing synopsis can weaken narrative comparison. TMDB can contain the same animated work as AniList, and anime seasons/sequels can crowd top matches. The baseline retains all rows, including potential same-work overlaps; source IDs cannot be equated across databases. Popularity-selected samples also limit generalization beyond these 2,000 records. Similarity measures the supplied text, not preference, quality, plot identity, or causal influence.

Primary references (accessed 11 October 2026).

[1] Qwen. Qwen3-Embedding-0.6B model card.

[2] Zhang et al. (2025). Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models.

[3] BAAI. BGE-M3 model card.

[4] Google. EmbeddingGemma model card and access terms.

[5] Sentence Transformers. Semantic Textual Similarity documentation.